

Aula 01 — Node.js: JavaScript sai do navegador

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Duração: 2h · **Pré-requisito:** ambiente configurado ([recursos/CONFIGURACAO-AMBIENTE.md](#))

Objetivos de aprendizagem

Ao final desta aula o aluno deve conseguir:

1. Explicar, com as próprias palavras, o que é o Node.js e por que ele existe.
2. Executar um arquivo `.js` pelo terminal com `node arquivo.js`.
3. Criar um projeto com `npm init` e entender para que serve o `package.json`.
4. Dividir o código em vários arquivos usando `export` e `import`.
5. Instalar e usar um pacote de terceiros com `npm install`.

Plano de aula (2h)

Tempo	Bloco	Conteúdo
0:00–0:10	Aquecimento	Boas-vindas ao módulo, o que vamos construir em 10 semanas (mostrar o projeto final rodando)
0:10–0:35	Conceito	O que é Node.js · runtime · onde o JS roda · npm e ecossistema
0:35–1:05	Código ao vivo	<code>node ola.js</code> → <code>process.argv</code> → <code>npm init</code> → <code>package.json</code> → módulos
1:05–1:15	Intervalo	
1:15–1:50	Mão na massa	Exercícios 1 a 3
1:50–2:00	Fechamento	Recapitulação, tarefa e prévia da aula 02

1. O que é o Node.js

Até o Módulo 0, todo o JavaScript que vocês escreveram rodava **dentro do navegador**. Ele estava preso ali: só conseguia mexer na página, ouvir cliques e mostrar `alert`.

O navegador tem, dentro dele, um programa chamado **motor JavaScript** — no Chrome ele se chama **V8**. É o V8 que lê seu código e o transforma em algo que o processador executa.

Em 2009, alguém teve uma ideia simples e poderosa: *"e se a gente pegasse esse motor e o colocasse para rodar sozinho, fora do navegador?"*. Isso é o **Node.js**.

NAVEGADOR	NODE.JS
Motor V8	Motor V8
+ document, window	+ acesso a arquivos
+ alert, fetch	+ criar servidor web
	+ acesso ao sistema
✗ não lê arquivos	✗ não tem document

A mesma linguagem, dois ambientes com poderes diferentes. `for`, `if`, funções e arrays são idênticos. O que muda é o que está disponível ao redor.

Analogia: o V8 é o motor de um carro. No navegador, esse motor está dentro de um carro de passeio (leva pessoas, tem banco e rádio). No Node, o mesmo motor foi colocado num trator (não tem banco de couro, mas puxa carga pesada e trabalha no campo).

Por que isso importa? Porque agora o mesmo desenvolvedor escreve o site e o servidor, na mesma linguagem. É o que torna possível o "full stack JavaScript" — que é exatamente o que vocês serão ao final deste módulo.

Onde o Node é usado de verdade

Netflix, Uber, LinkedIn, PayPal, Nubank (em partes), praticamente toda startup moderna. Não é uma ferramenta de curso — é a ferramenta com que você vai trabalhar.

2. Rodando o primeiro arquivo

Crie um arquivo `ola.js`:

```
console.log("Olá, mundo do back-end!");
```

No terminal, **dentro da pasta onde o arquivo está**:

```
node ola.js
```

Isso é tudo. Não existe HTML, não existe navegador, não existe `<script>`. O Node lê o arquivo, executa de cima para baixo e termina.

Erro nº 1 da aula: `Error: Cannot find module '/caminho/ola.js'`. Traduzindo: "você me pediu para rodar um arquivo que não existe aqui". Rode `ls` (Windows: `dir`) e confira se `ola.js` aparece na lista. Se não aparece, você está na pasta errada.

O terminal interativo (REPL)

Digite só `node` e aperte Enter. Você entra num modo onde pode testar JavaScript linha a linha — igual ao console do navegador. Ótimo para tirar dúvidas rápidas. Saia com `.exit` ou `Ctrl+C` duas vezes.

3. Recebendo dados de fora: `process.argv`

Um programa que sempre faz a mesma coisa é pouco útil. No navegador você pegava dados de um `<input>`. No terminal, você pega dos **argumentos da linha de comando**.

```
// exemplos/02-argumentos.js
const nome = process.argv[2];
console.log(`Olá, ${nome}!`);
```

```
node 02-argumentos.js Maria
# Olá, Maria!
```

`process` é um objeto que o Node oferece com informações sobre a execução atual. `process.argv` é um array onde:

Posição	Conteúdo
<code>[0]</code>	caminho do executável do Node
<code>[1]</code>	caminho do seu arquivo
<code>[2]</code> em diante	o que você digitou

Por isso os dados úteis começam no índice `2`.

4. `npm init` e o `package.json`

Um arquivo solto é um script. Um **projeto** precisa de identidade: nome, versão, lista de dependências e comandos. Isso é o `package.json`.

```
mkdir meu-projeto
cd meu-projeto
npm init -y
```

O `-y` responde "sim" para tudo e gera o arquivo direto. Resultado:

```
{
  "name": "meu-projeto",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  }
}
```

Vamos ajustar duas coisas — e essa é a **configuração padrão de todos os projetos do curso**:

```
{
  "name": "meu-projeto",
  "version": "1.0.0",
  "type": "module",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "node --watch index.js"
  }
}
```

- **"type": "module"** → habilita `import / export`, o padrão moderno da linguagem. Sem isso, o Node reclama: `Cannot use import statement outside a module`.
- **scripts** → apelidos para comandos. Agora `npm start` roda `node index.js`.
- `npm run dev` usa `--watch`: o Node reinicia sozinho toda vez que você salva o arquivo. Você vai usar isso o módulo inteiro.

5. Módulos: dividindo o código em arquivos

Colocar tudo em um arquivo funciona até ele ter 300 linhas. Depois vira caos. A solução é **separar por responsabilidade**: cada arquivo faz uma coisa e diz o que oferece ao mundo.

`saudacao.js` — exporta duas funções:

```
export function saudar(nome) {  
  return `Olá, ${nome}!`;  
}  
  
export function despedir(nome) {  
  return `Até logo, ${nome}!`;  
}
```

`index.js` — importa e usa:

```
import { saudar, despedir } from "./saudacao.js";  
  
console.log(saudar("Turma FPA"));  
console.log despedir("Turma FPA"));
```

Três regras que evitam 90% dos erros:

1. O caminho começa com `./` quando é um arquivo **seu**. Sem `./`, o Node procura em `node_modules`.
2. A **extensão** `.js` é **obrigatória** no import. No navegador e em outras ferramentas às vezes não é — no Node com ESM, é.
3. Só sai do arquivo o que tem `export`. O resto é privado — e isso é uma vantagem, não uma limitação.

`export default`

Quando o arquivo exporta **uma coisa principal**, use `default`:

```
// calculadora.js  
export default function somar(a, b) {  
  return a + b;  
}  
  
// index.js — sem chaves, e o nome é escolhido por quem importa  
import somar from "./calculadora.js";
```

Regra prática do curso: use `export` **nomeado por padrão**. Ele deixa explícito o que veio de onde e o editor autocompleta melhor. Reserve o `default` para quando o arquivo tem um propósito único.

E o `require`?

Você vai encontrar muito código na internet assim:

```
const saudacao = require("./saudacao"); // jeito antigo (CommonJS)
module.exports = { saudar };
```

É o **CommonJS**, o sistema antigo do Node. Funciona, mas não é mais o padrão. Saiba **reconhecer** para não se confundir ao ler tutoriais antigos — e escreva `import / export` no seu código.

6. npm: usando código dos outros

O npm é o maior repositório de código aberto do mundo — mais de 2 milhões de pacotes. Antes de escrever qualquer coisa complicada, vale perguntar: *"alguém já resolveu isso?"*

```
npm install chalk
```

O que acontece:

1. O npm baixa o pacote (e as dependências dele) para a pasta `node_modules/`.
2. Registra o pacote em `dependencies`, dentro do `package.json`.
3. Cria/atualiza o `package-lock.json` com as versões exatas.

```
import chalk from "chalk";

console.log(chalk.green("✓ deu certo"));
console.log(chalk.red("✗ deu errado"));
```

Três coisas para memorizar

Arquivo/pasta	Vai para o Git?	Por quê
<code>package.json</code>	✅ sim	É a receita do projeto
<code>package-lock.json</code>	✅ sim	Garante que todos instalem a mesma versão
<code>node_modules/</code>	❌ nunca	Pesa centenas de MB e é reconstruível com <code>npm install</code>

Por isso todo projeto tem um `.gitignore` com `node_modules/` dentro. Se você recebe um projeto sem a pasta `node_modules`, basta rodar `npm install` e ela volta inteira.

7. Ferramentas novas desta aula

Toda aula termina com este quadro: as ferramentas nativas do JavaScript que apareceram pela primeira vez. A explicação completa de cada uma está em [recursos/METODOS-NATIVOS.md](#) — use como dicionário.

`console.log()`

O que faz: escreve uma mensagem no terminal. **Quando usar:** para você acompanhar o que acontece dentro do programa.

```
console.log("texto");
console.log(42);
console.log({ nome: "Ana" }); // mostra o objeto inteiro
console.log("a", "b", 42); // aceita vários valores, separados por vírgula
```

A família completa: `console.error()` (erros), `console.warn()` (avisos) e — muito útil e pouco conhecida — `console.table()`, que imprime um array de objetos como tabela formatada.

⚠ **`console.log` não é `return`**. Esta é a confusão nº 1 de quem vem do Módulo 0:

```
function somar(a, b) { console.log(a + b); } // mostra na tela, devolve undefined
function somar(a, b) { return a + b; }      // devolve o valor para quem chamou
```

`console.log` é para **você** ver. `return` é para o **programa** usar.

Template literals (as crases)

O que faz: monta um texto com variáveis dentro. **Quando usar:** sempre que misturar texto com valores.

```
const nome = "Maria";

"Olá, " + nome + "!" // jeito antigo
`Olá, ${nome}!`      // jeito moderno ✅
```

Dentro de `${...}` cabe qualquer **expressão** — não só variável:

```
`Nasceu em ${2026 - idade}`
`Status: ${idade >= 18 ? "adulto" : "menor"}`
```

⚠ É **crase** (```), não aspas simples. No teclado ABNT2, fica ao lado do `P` — aperte a tecla e depois a barra de espaço.

`process` — o objeto do Node

`process` é um objeto que o Node oferece com informações sobre a execução atual. Ele **não existe no navegador** — é um dos "poderes a mais" de que falamos no começo da aula.

Propriedade	O que é
<code>process.argv</code>	array com o que foi digitado na linha de comando
<code>process.exit(codigo)</code>	encerra o programa imediatamente (<code>0</code> = ok, outro = erro)
<code>process.env</code>	variáveis de ambiente (veremos na aula 06)

⚠ `process.argv` sempre entrega texto, mesmo quando parece número:

```
// node soma.js 10 3
const a = process.argv[2];      // "10" - é TEXTO
a + process.argv[3]             // "103" 🗑 concatenou

Number(a) + Number(process.argv[3]); // 13 ✅
```

`Number(valor)` converte texto em número. Vamos usá-la o módulo inteiro, porque **tudo que vem de fora chega como texto** — do terminal, da URL, do formulário.

`.length`

O que faz: informa o tamanho de um texto ou de um array.

```
"Maria".length      // 5
[10, 20, 30].length // 3
```

⚠ É uma **propriedade**, não um método: **não tem parênteses**. `"Maria".length()` dá erro. Regra: método executa algo (tem parênteses); propriedade só é lida.

Recapitulação em 3 frases

1. Node.js é o motor do JavaScript rodando fora do navegador, com poderes de servidor.
2. `package.json` é a identidade do projeto; `npm install` traz código de terceiros; `node_modules` nunca vai para o Git.
3. `export` mostra o que o arquivo oferece, `import` traz — sempre com `./` e `.js`.

Tarefa para a próxima aula

Exercícios 1 a 3 de [exercicios/README.md](#). O desafio é opcional, mas tente.

Na próxima aula

Vamos afiar o JavaScript que profissionais realmente escrevem: `map`, `filter`, `reduce`, desestruturação e spread. É a base de tudo que faremos nas próximas 9 semanas.
